ADDRESS THE MESS

A 30-Day Process
to Identify Ownership and Safely Change Systems

A 30-Day Forcing Function for Practitioners in Messy Environments

By Djerhy Jn Baptiste

IS THIS FRAMEWORK FOR YOU?

The 90-Second Mess Test

Before investing 30 days, confirm this addresses your actual problem. Answer honestly:

□ When an auditor asks "who owns this system?"

does it take more than 5 minutes to get an answer?

□ Have you discovered production systems running that no one remembers deploying?

□ Do you have assets labeled "test" or "dev" that are over 6 months old?

□ Has a vendor deprecation notice ever caused an unexpected outage?

□ Do former employees still have active credentials 30+ days after departure?

□ Can you name who has authority to decommission your three most critical assets?

□ Have you paid for software licenses you're not using in the last 12 months?

□ Does your e-waste pile include servers you're "not sure we can turn off"?

SCORING:

0-2 yes: Your governance is rare. This framework is insurance.

3-5 yes: You have the typical mess. This framework will deliver ROI.

6-8 yes: You're in crisis debt. Start Week 1 today.

If you checked 3 or more boxes, you don't have an inventory problem. You have an authority problem.

WHAT THIS IS NOT

This is not a maturity assessment.
There is no scoring model or capability rating.

This is not a transformation program.
It is a short, contained effort focused on one system.

This is not tool selection or consulting guidance.
You will work only with the environment you already have.

This is not a best-practice framework.
The goal is defensible operational understanding.

This is not comprehensive.
It is a limited process to establish whether control and ownership can be demonstrated.

PART I: WHY VISIBILITY DOESN'T MEAN CONTROL

Lists, Logs, Lies

Discovery tools are good at showing what exists. They are far less reliable at showing who can act. Logs accumulate, inventories grow, and dashboards fill with signals, but none of this guarantees that anyone has the authority to intervene when something needs to change.

Dashboards without decision rights create passive observation. They show activity, trends, and anomalies, but stop short of enabling action. Without the authority to intervene, visibility becomes descriptive rather than governing.

Compatibility is often treated as permanent until it fails publicly. Assets remain on the books long after the conditions that made them viable have expired. As backward compatibility quietly erodes, "known" systems turn into operational surprises. Inventories stay static while contextual viability degrades underneath them.

You Can See It, But You Can't Stop It Fast Enough

Modern systems are designed for velocity, not hesitation. Decision friction wasn't removed deliberately. It was optimized away in the pursuit of throughput and efficiency. Informal safeguards that once slowed bad decisions were replaced by systems designed to act first and reconcile later. Speed was preserved, but authority failed to keep pace.

Infrastructure defined by code now deploys at a speed human review can't realistically match. Even competent, well-staffed teams are outpaced by automated changes driven by templates, pipelines, and vendor defaults. Over time, authority shifts quietly to third-party logic, optimization engines, auto-scaling rules, and external algorithms, without explicit organizational consent.

The Authority Vacuum

"Responsible" does not mean "empowered to decide." Many systems are visible but effectively untouchable. They run, incur cost, and carry risk, yet no one can confidently pause them, change them, or end them.

Assets accumulate when people are responsible for upkeep but lack the authority to end them. E-waste
is the physical residue of decisions no one was empowered to reverse. Systems persist not because they are needed, but because decommissioning feels riskier than continued neglect. What "still works" slowly becomes "still here."

Failure mode defined:

The problem is not speed. It is authority lag, decision rights failing to keep pace with operational reality.

Core Thesis (Part I)

You can't govern what you can't act on.
Most organizations have visibility.
Almost none have enforceable authority.

PART II: THE ASSET INTELLIGENCE METHOD™

Seeing Enough to Decide

Asset intelligence is not about seeing everything. It is about seeing enough to make decisions that hold under pressure. That requires tracking not only assets, but the invisible connective tissue that gives them meaning: dependencies, protocols, and integration contracts. Compatibility is borrowed time (as established earlier), and dependency intelligence makes that erosion visible before it becomes an incident.

Application: Dependency Intelligence

Input: Asset plus dependency surface
Decision: Can this be paused, changed, or ended safely?
Action: Informed approval or escalation

Ownership Means Authority, Not Paperwork

Ownership defines accountability, but authority determines whether that accountability can be exercised under pressure. When ownership exists without authority, it collapses into administrative labeling. Authority is what converts ownership from a record into an executable decision.

Every asset must have clear authority boundaries. What can be approved unilaterally? What requires escalation? How long can a decision reasonably wait?

Test: If no one can approve taking it offline for ten minutes, the asset is unowned.

Application: Authority Mapping

Input: Named owners and decision scope
Decision: Who can authorize change or deletion?
Action: Executable accountability

Context Exists to Enable Action Enrichment is not about completeness. It is about decisiveness.

Enrichment is not about completeness. It is about decisiveness.

Minimum Viable Enrichment (MVE) is the smallest amount of context required to make a defensible decision under pressure. Enrichment stops at the point where action becomes possible. Anything beyond that adds documentation, not control.

MVE includes:
- Ownership
- Classification
- Exposure
- Authority level

The only question that matters:
What breaks if this disappears, and who is empowered to make that call?

Application: Decision-Grade Asset Registry

Core Thesis (Part II)

Governance that only works on paper collapses under real conditions.
If it fails the moment speed, risk, or uncertainty enters the system, it was never control. It was documentation masquerading as governance.

THE INTENDED OUTCOME

This process produces one deliverable: an internal operational summary.

At the end of the 30 days, you should be able to hand a short document to your manager or team lead that answers four questions about a real system:

- Who is accountable for it
- Who has access to it
- What depends on it
- What change would be risky

if you can provide those answers, the process has succeeded.

The goal is not analysis or documentation for its own sake.
The goal is to move from uncertainty to a defensible understanding that others can rely on.
The pages that follow are simply the steps needed to produce that summary safely.

PART III: THE 30-DAY ACCOUNTABILITY SPRINT

Purpose:
Prove your organization can identify, decide, and delete.
This is not transformation. It is a capability test.

Week 1: Map What You Actually Control

- Floor walk (physical verification)
- Select three asset classes (e.g., servers, SaaS apps, service accounts)
- For each:

1. Who can authorize downtime?
2. Who can approve decommissioning?
3. Who can revoke access?

- Flag authority gaps

Deliverable: Authority map for three critical asset classes—enough to expose where decision rights have collapsed and where interim custody is required.

Week 2: Assign Authority and Test It

- Assign interim owners where gaps exist
- Select one zombie asset:
  Test or dev system
  Unused license
  Orphaned service account
- Decommission it within seven days
- Document:
  What broke
  Who objected
  What was learned

Deliverable: Proof you can delete

Week 3: Build the Deletion Pipeline

- Create a real decommissioning workflow (not a "sunset plan")
- Identify the next five removal candidates
- Validate dependencies
- Secure owner sign-off

Deliverable: Deletion pipeline with named approvers

Week 4: Operationalize and Report Out

-
- Execute two to three additional deletions
- Quantify ROI:
  Licenses canceled
  Contracts ended
  Power or space reclaimed
- Present results:
  "We ended X"
  "We saved Y"
  "We learned Z"
- Establish monthly cadence:
  Ownership validation
  Deletion review

Deliverable: Repeatable process with executive-level proof

Core Thesis (Part III)

Governance that cannot delete is theater.
Thirty days is enough to prove whether control exists or not.

PART IV: WHERE AUTHORITY AND VELOCITY COLLIDE

Unifying pattern:
Each domain below fails the same way, authority becomes separated from visibility in high-speed environments.

If It Draws Power, It Draws Risk

- Physical Assets and E-Waste
- Floor walk methodology
- Physical-to-logical reconciliation
- E-waste as governance evidence, not sustainability theater

If authority breaks down in the most tangible environment (physical space), it's almost certainly broken in abstract ones.

Leaving Access Active
Exit, Closure, and M&A

- Pre-deal asset hygiene
- Post-merger authority reconciliation
- Zombie detection: "retired" systems still running

Even when systems are deliberately retired, authority gaps persist. When systems are never meant to retire (like automation accounts) the problem compounds.

Bots Don't Wait for Approval
Non-Human Identity Governance

- Service accounts, API keys, automation actors
- NHI-to-asset linkage
- Lifecycle and termination authority for non-human decision-makers

Non-human identities operate with organizational authority but no accountability structure. They act continuously, and most organizations discover this only during incidents or audits.

Models Act as Your Organization
AI System Authority Governance

- Treat model deployment like executive onboarding
- Define scope, limits, accountability, and termination criteria
- Governance at inference speed

When models make decisions at machine speed, traditional approval workflows become impossible. Authority must be encoded upfront, not applied retroactively.

External Was Treated as a Location, Not an Asset Class Vendor and Third-Party Authority

- Vendor-connected asset inventory
- Compatibility withdrawal monitoring
- Shadow SaaS as ungoverned authority

External systems often hold more operational authority than internal ones. They can break you, but you can't govern them the same way. The boundary isn't location, it's impact.

Core Thesis (Part IV)

The method does not change.
Only the speed and blast radius do.

PART V: TOOLS, TEMPLATES, AND PROOF

You can read about maturity models, or you can prove you've moved up one. Here's how to measure what matters. Templates That Drive Decisions The following templates are designed for immediate use. Copy, adapt, deploy.

TEMPLATE 1: AUTHORITY MAPPING WORKSHEET

```
ASSET: ______________________
ASSET CLASS: □ Physical □ Virtual □ SaaS □ NHI □ Vendor-Connected

OWNERSHIP & AUTHORITY

├── Technical Owner: ______________
│   Can approve: □ Downtime (< 10 min) □ Config changes □ Decommissioning
│   Requires escalation for: ______________
│
├── Business Owner: ______________
│   Can approve: □ Contract cancellation □ Budget reallocation □ Feature sunset
│   Requires escalation for: ______________
│
└── Data Steward: ______________
    Can approve: □ Access revocation □ Data deletion □ Export/migration
    Requires escalation for: ______________

DECISION SPEED TEST
Can this asset be taken offline for 10 minutes with < 1 hour notice?
□ Yes → Owner: ______________
□ No → Escalation path: ______________
□ Unknown → INTERIM CUSTODY REQUIRED

LAST VALIDATED: ___/___/___
NEXT REVIEW: ___/___/___
```

TEMPLATE 3: DECOMMISSIONING ROI CALCULATOR

Asset: _______________
Annual cost to maintain: $_______________

COST BREAKDOWN:
├── License/subscription: $_______________ /year
├── Support contract: $_______________ /year
├── Power/cooling/space: $_______________ /year
├── Personnel time (hrs/week × hourly rate): $_______________ /year
└── Risk/compliance overhead: $_______________ /year

TOTAL ANNUAL COST: $_______________

Expected lifespan if not decommissioned: ___ years
TOTAL COST AVOIDED: $_______________

Decommissioning effort: ___ hours × $___ /hour = $_______________
NET ROI: $_______________ (Total Avoided - Decommissioning Cost)

TEMPLATE 4: ZOMBIE ASSET IDENTIFICATION CHECKLIST

Run this checklist against any suspect asset:

□ No one can name the business owner within 5 minutes
□ Labeled "test", "dev", "temp", "old", "backup", or contains a date in the name
□ Last modified/accessed > 90 days ago
□ No monitoring alerts configured
□ Not referenced in any disaster recovery plan
□ Original deployer has left the organization
□ Running an unsupported OS or software version
□ No documentation exists explaining its purpose
□ No budget line item explicitly funds it □ When asked "can we turn this off?", the answer is "probably, but..."

SCORING:
0-2 checks: Likely legitimate, verify ownership
3-5 checks: Zombie candidate, investigate immediately
6-10 checks: Zombie confirmed, schedule decommissioning

TEMPLATE 5: MVE ENRICHMENT SCHEMA

MINIMUM VIABLE ENRICHMENT (MVE)
The smallest context required to make a defensible decision.

ASSET ID: _______________
NAME: _______________
TYPE: □ Physical □ Virtual □ SaaS □ NHI □ Vendor

OWNERSHIP (Required)
├── Technical Owner: _______________
├── Business Owner: _______________
└── Data Steward: _______________

CLASSIFICATION (Required)
├── Criticality: □ Critical □ Important □ Standard □ Low
├── Data Sensitivity: □ Public □ Internal □ Confidential □ Restricted
└── Environment: □ Production □ Staging □ Development □ Test

EXPOSURE (Required)
├── Internet-facing: □ Yes □ No
├── Processes PII/PHI/PCI: □ Yes □ No
└── External dependencies: _______________

AUTHORITY LEVEL (Required)
Who can authorize decommissioning: _______________
Approval required for downtime: _______________
Maximum decision latency: ___ hours/days

STOP ENRICHING HERE unless additional context changes the decision.

Maturity Model: Can You Delete?

Level 1: Chaotic — no one knows who can end things
Level 2: Reactive — deletion only happens in crisis
Level 3: Managed — documented ownership and authority
Level 4: Measured — deletion velocity is tracked
Level 5: Optimized — proactive decommissioning is routine

Most organizations enter at Level 2. Movement to Level 3 requires the 30-day sprint.

Metrics That Matter

- Mean time to identify owner: < 15 minutes
- Percentage of assets with documented authority: > 95%
- Deletion velocity: assets decommissioned per quarter
- Authority gap incidents: zero

APPENDICES

Anti-Patterns That Break AddressTheMess

- Treating Lean as a service instead of subtraction
- Confusing "still works" with "still supported"
- Assigning ownership without decision rights
- Measuring activity instead of deletion

Why Teams Hesitate (And Why They're Wrong)
"We don't have time for a 30-day sprint."

You're already spending time explaining to auditors why you can't answer basic questions. The sprint costs 4-6 hours per week. The next audit finding will cost you 40+ hours in remediation plus reputational damage.

"We need executive buy-in first."

Week 1 doesn't require executive approval. It requires a floor walk and three authority conversations. If your execs won't approve a floor walk, you have bigger problems than asset governance.

"Our environment is too complex for this."

Complexity is the reason you need this, not the reason it won't work. The sprint doesn't require perfect documentation, it requires one deletion. If you can't decommission a single test server in 7 days, your governance is broken.

"What if deleting something breaks production?"

Week 2 explicitly says: document what broke. If nothing broke, your zombie detection is accurate. If something broke, you've just discovered undocumented dependencies, which is exactly the visibility problem this framework addresses.

"We already have an asset management system."

You have a database. This framework determines whether anyone has authority to act on what's in it. Most CMDB/ITAM tools tell you what exists. This tells you who can end it.

APPENDICES

Anti-Patterns That Break AddressTheMess

- Treating Lean as a service instead of subtraction
- Confusing "still works" with "still supported"
- Assigning ownership without decision rights
- Measuring activity instead of deletion

Why Teams Hesitate (And Why They're Wrong)

"We don't have time for a 30-day sprint."

You're already spending time explaining to auditors why you can't answer basic questions. The sprint costs 4-6 hours per week. The next audit finding will cost you 40+ hours in remediation plus reputational damage.

"We need executive buy-in first."

Week 1 doesn't require executive approval. It requires a floor walk and three authority conversations. If your execs won't approve a floor walk, you have bigger problems than asset governance.

"Our environment is too complex for this."

Complexity is the reason you need this, not the reason it won't work. The sprint doesn't require perfect documentation, it requires one deletion. If you can't decommission a single test server in 7 days, your governance is broken.

"What if deleting something breaks production?"

Week 2 explicitly says: document what broke. If nothing broke, your zombie detection is accurate. If something broke, you've just discovered undocumented dependencies, which is exactly the visibility problem this framework addresses.

"We already have an asset management system."

You have a database. This framework determines whether anyone has authority to act on what's in it. Most CMDB/ITAM tools tell you what exists. This tells you who can end it.

"We can't get stakeholder alignment."

Week 0 provides the exact language to use. If you can't get 15 minutes with four people to explain a 30-day experiment, your organization has deeper dysfunction than asset governance can solve.

"What if we identify problems we don't have resources to fix?"

Then you've documented risk leadership chose to accept. That's governance. Documenting what you can't fix is infinitely better than not knowing what's broken.

One-Paragraph Summary

Most organizations know what they have. Almost none know who can act on it. AddressTheMess restores control by reconnecting visibility to decision authority—through ownership with teeth, dependencies that aren't invisible, and proven ability to end systems safely. This is not a transformation roadmap. It is a 30-day forcing function that proves whether governance is real or merely well-documented theater.

WHAT'S NEXT

If you scored 3+ on the Mess Test, you don't need permission to start.

You need:
A calendar with 4-6 hours blocked per week
Access to three asset classes
One zombie asset
Seven days Week

1 starts Monday.

For additional resources, templates, and community discussion:
djerhy.tech/addressthemess